NATIVE AI SOFTWARE & SYSTEMS ENGINEERING FOR UAVS

UAV Infrastructure

Stage 1 — ArduPilot SITL, an MQTT contract, and your first drone frontend.

▸ STAGE 1 OF 6 · pymavlink ⇔ MQTT plumbing

// FOUNDATIONS

What is ArduPilot?

The open-source autopilot **firmware** that flies the aircraft — not a simulator, not a toy, the real flight-control stack.

- **Real flight-control firmware** — runs the actual control loop: attitude stabilization, navigation, failsafes. The exact binary you'll run in SITL this semester is what flies real aircraft.
- **Broad platform support** — multicopters, fixed-wing, rovers, submarines, all from one open-source project with a long track record (in active development since 2007).
- **Production-grade, not academic** — used across industry, research labs, and hobbyist communities worldwide, with a large contributor base and active safety-critical development.

// A DELIBERATE CHOICE

Why ArduPilot, not PX4?

Both are excellent, mature, open-source autopilot stacks. The decision here isn't about which is "better."

THE DECIDING FACTOR

Your real drones already fly ArduPilot

Code, MAVLink message handling, and workflows you build against SITL this semester carry over directly to the physical hardware later — same firmware family, same parameters, same behavior.

→

NOT A KNOCK ON PX4

Deployment-target consistency wins

PX4 is a strong, actively developed alternative used widely in research. If your target hardware ran PX4, this course would too — the choice tracks the aircraft, not a technical preference.

// FOUNDATIONS

What is SITL?

Software-In-The-Loop: the real ArduPilot flight-code binary, running against a simulated physics model instead of real motors and sensors.

- **No hardware, no risk** — safe to crash, safe to experiment, safe to make the mistakes you'll learn from.
- **Not a toy simulator** — it's the identical firmware that runs on the physical flight controller, just fed simulated sensor data instead of real ones.
- **Why not Gazebo?** Our SITL uses ArduPilot's own lightweight built-in physics model, not a 3D physics/rendering engine. Gazebo needs real GPU-accelerated OpenGL, and in a prior run of this kind of exercise that broke unpredictably across a room of student laptops — opaque failures, exactly what derails a live class session. ArduPilot's internal model is enough for GPS, attitude, battery, and mission logic; Gazebo stays opt-in for a team that specifically needs camera or collision simulation later.

what you'll actually see — GCS connected to SITL

Gazebo — not used by default (GPU/OpenGL dependency)

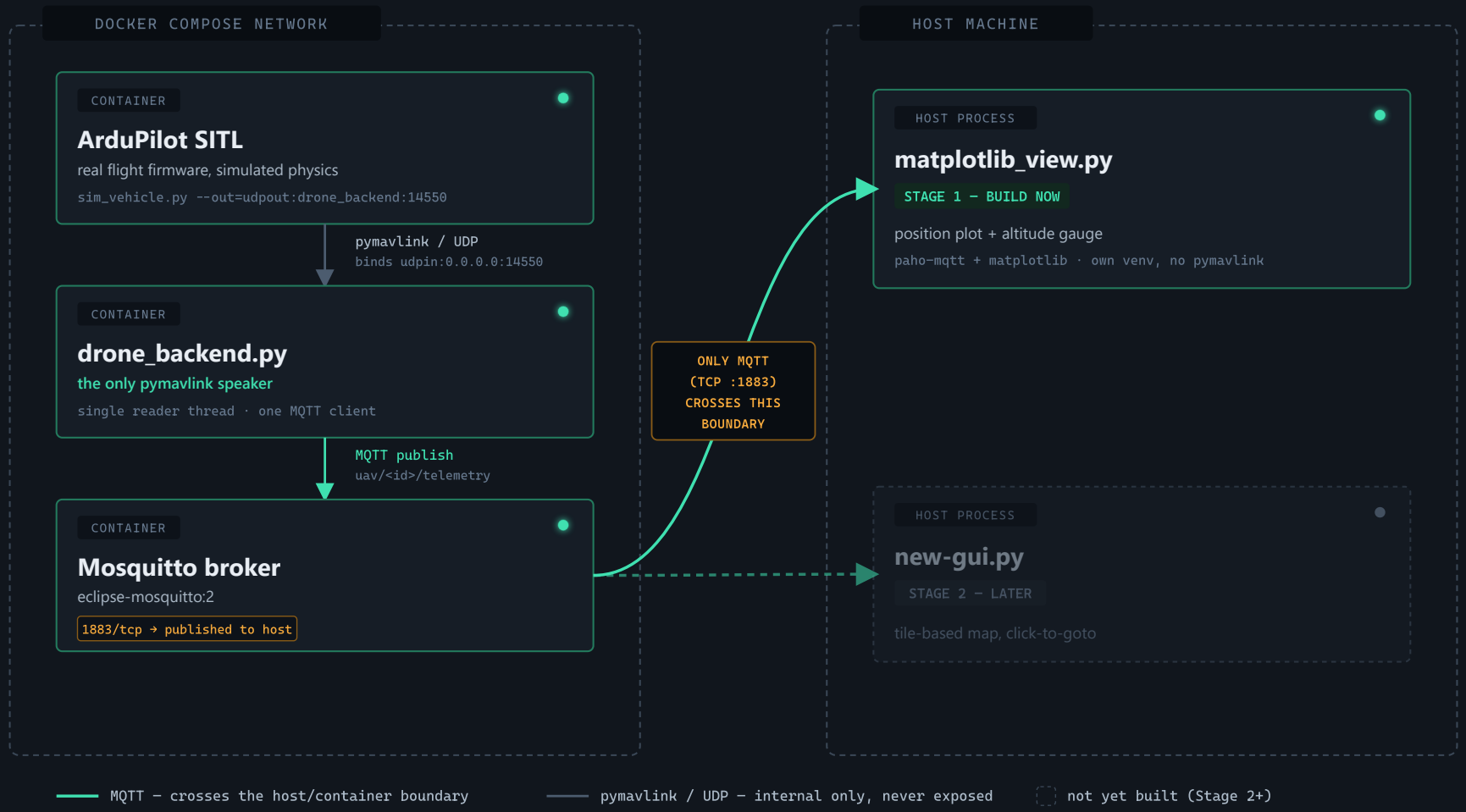
// THE REAL THING

Meet your (future) real hardware

Everything this week runs in simulation. This is what your code eventually flies on.

// ARCHITECTURE

The big picture

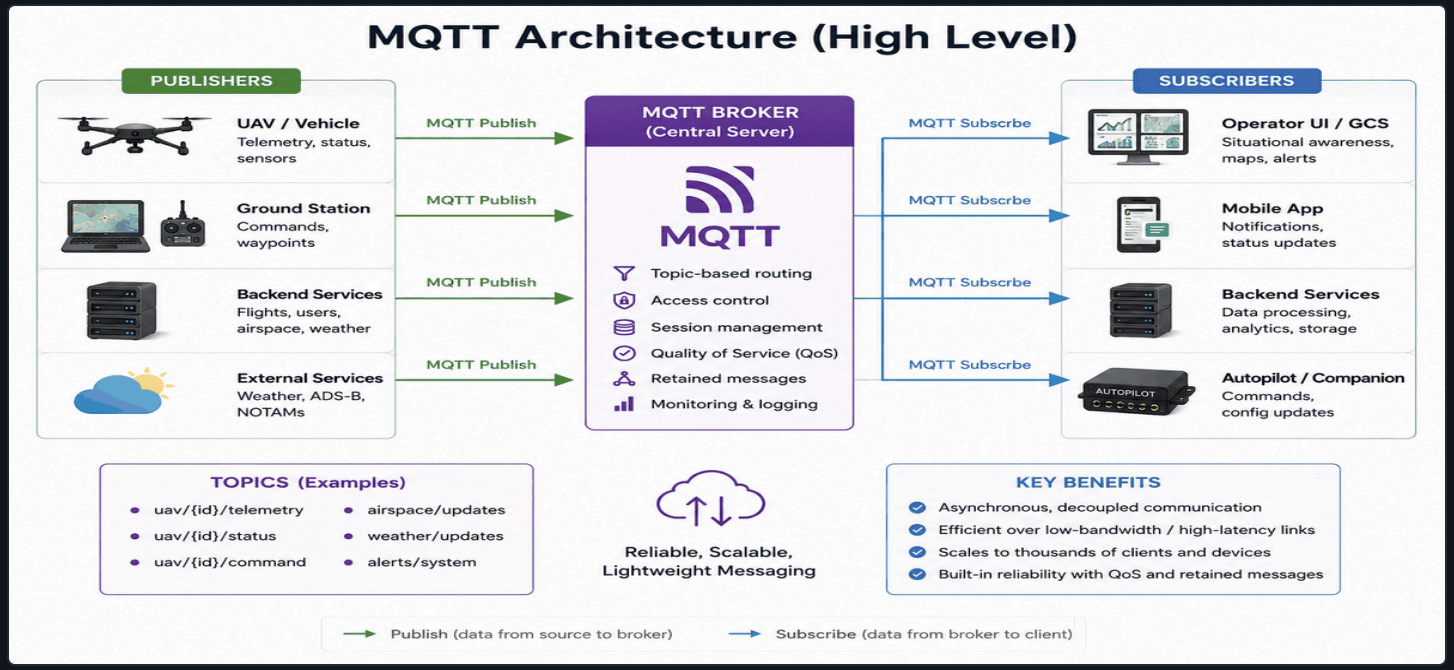


// ARCHITECTURE

Why MQTT as the interface?

The topic schema **is** the contract — a clean, concrete example of interface design, not just plumbing.

- **Multiple frontends, one shared contract** — many students or teams can build against the same documented topics without touching the backend.
- **Swappable by design** — replacing matplotlib with a real map-based GUI later (Stage 2) requires **zero** backend changes.
- **A lesson in its own right** — this decoupling is exactly the kind of interface-design decision you'll be asked to make and defend in real systems.



generic MQTT topology — reference

// ARCHITECTURE

The MQTT contract — real topics, real JSON

→ uav/1/telemetry

```
{
  "vehicle_id": "1",
  "lat": 41.700, "lon": -86.239,
  "alt_rel": 12.4, "heading": 87.3,
  "armed": true, "mode": "GUIDED"
}
```

→ uav/1/command

```
// one of:
{"type": "arm"}
{"type": "takeoff", "alt": 10}
{"type": "goto", "lat":…, "lon":…}
{"type": "land"}
```

→ uav/1/home RETAINED

```
{
  "vehicle_id": "1",
  "lat": 41.700,
  "lon": -86.239,
  "alt": 225.1
}
```

Published once, **retained** by the broker — any client subscribing later still gets it immediately, including clients that join mid-flight.

ARCHITECTURAL TRADEOFF

Retain makes the broker hold real state, not just relay it — is that MQTT's job?

// A SUBTLETY IN THE CONTRACT

Coordinate frames: LLA vs. local ENU

ON THE WIRE

LLA is canonical

Lat / lon / alt is the only frame that ever crosses a system boundary — it matches how MAVLink represents position natively, and stays stable as more clients join.

INSIDE ONE CLIENT

Local ENU is private & throwaway

Metres-from-an-origin math a client computes for its own use (e.g. a legible 2D plot). Never published, never shared truth between clients.

Why it matters: two clients quietly disagreeing about where "local (0,0)" is produces a bug class that's miserable to debug — correct in one view, subtly wrong in another. The retained → `uav/<id>/home` topic fixes this — one shared origin, delivered to every client the moment it subscribes.

LLA — geodetic

lat, lon, alt · WGS-84

Global, Earth-fixed. What GPS and MAVLink speak natively — this project's wire format.

ENU — local tangent

East, North, Up · E×N=U

Local, per-client, right-handed. Common in robotics, SLAM/VIO, and mapping.

NED — aerospace

North, East, Down · N×E=D

ArduPilot/PX4's internal convention for attitude and flight dynamics — not what we publish, but what's under the hood.

// INFRASTRUCTURE

Why Docker — but not your Python code

The obvious layout — backend on the host, SITL in Docker — was the original plan. It didn't survive contact with real networking.

WHAT BROKE

Host ↔ container UDP is genuinely ambiguous

MAVProxy's `--out` is client-mode (`udpout`), pymavlink's bare `udp:` prefix has shifted bind/connect semantics across versions, and Docker Desktop vs. native Linux/WSL2 disagree about `host.docker.internal`.

THE FIX

Same Compose network, service-name DNS

`drone_backend` binds `udpin:0.0.0.0:14550` (listens). SITL launches with `--out=udpout:drone_backend:14550` (connects out, by name). No port published for this link at all — only MQTT crosses the host boundary.

General rule this leaves you with: containerize infrastructure nobody needs to touch (version-pinned, reproducible — "does this work" reduces to "does Docker run"). Keep what you and Claude Code edit constantly — `drone_backend.py`'s source is bind-mounted, and your viewer runs straight on the host — so no rebuild-and-restart cycle gets added to routine editing.

// THIS SESSION

matplotlib_view.py — the viewer you're about to build

WHY A MINIMUM VIEW SPAN

MIN_HALF_SPAN_M = 20

Raw autoscale fits tightly to whatever data exists — including a few centimeters of GPS/EKF noise while sitting still, which visually blows up into what looks like a wild flight. The floor only grows once the vehicle actually flies further.

WHY A SHORT TRAIL

TRAIL_LENGTH = 20 (~5s @ 4Hz)

Reads as a comet-tail trailing the vehicle, not the whole mission's path slowly aging out over a full minute.

WHY A TRIANGLE, NOT AN ARROW

heading_triangle() marker

Replaced an earlier circle + quiver-arrow combination — a single rotating shape reads heading faster at a glance.

WHY ONE SHARED COLOR CONSTANT

DRONE_COLOR

Marker, trail, and altitude gauge all pull from one constant — so a future second vehicle (Stage 6) reads as one consistent visual signature, not just a differently-colored dot.

// BEFORE YOU FLY

Known quirks worth knowing

None of these are bugs in your code. They'll look like it if you don't know them going in.

- **"mode" doesn't reset after landing.**
  ArduCopter leaves `mode: "LAND"` reported indefinitely after touchdown and disarm — it only changes on the next explicit mode switch. Check `armed` too if "currently landing" matters.

- **Pre-arm checks can reject arm/takeoff for 30–60s after cold boot.**
  EKF/GPS not settled yet, or "Gyros inconsistent." Expected, not a bug — handle it with bounded retries, not by failing immediately.

- **DISARM_DELAY auto-disarms an idle armed vehicle (~10s).**
  Mainly bites during manual MAVProxy console testing — arm and issue your next command in one quick burst, or it disarms out from under you.

// INFRASTRUCTURE

Getting SITL without a 10–20 minute build

ArduPilot builds from source by default. Nobody should have to sit through that on the first day of class.

- **Pre-built and pinned** — `Copter-4.6.3`, built once from `ardupilot-dev-base`, pushed to a public GHCR image. You run `docker compose pull`, not a source build.
- **Apple Silicon note** — the image is `linux/amd64` only (no arm64 build exists upstream). It runs fine under Docker Desktop's Rosetta-accelerated emulation — one-time setting, covered in SETUP.md.
- **Bumping the version later** is one script and one `.env` line — nothing else in the repo changes.

// THIS SESSION

What you'll actually do

01 **Start SITL, then Mosquitto**
Skip the built-in map GUI while debugging — one less window competing for attention.

02 **Build drone_backend.py, verify in isolation**
Confirm telemetry with `mosquitto_sub -t 'uav/#' -v` before writing any client.

03 **Build matplotlib_view.py, confirm it moves**
Fly SITL manually first to check the viewer reflects real telemetry.

04 **Wire up commands last**
Arm/takeoff/goto/land over MQTT — isolating variables, one layer at a time.

// GETTING STARTED

Run one command

Not a setup guide to follow by hand — a single script that tells you pass/fail in plain English.

```
$ ./scripts/verify_setup.py

PASS: Docker is installed and the daemon is running.
PASS: docker compose up succeeded.
PASS: telemetry flowing (...)
PASS: command round-trip works (arm command was obeyed).

PASS: environment is fully working.
```

If it fails, it tells you exactly what broke and what to do next — not a stack trace. Stuck for more than a few minutes? Pair with your neighbor before flagging it down — most classroom setup failures are one of a small, known set.

scripts/test_flight.py runs the same idea end-to-end: arm → takeoff → small square → land, entirely over MQTT. Each command re-publishes every 8s until its telemetry-based success condition is met — so it self-heals against a dropped UDP command or the cold-start pre-arm flakiness above, instead of just failing once.